Lab Sheet-05
BCA201P
Foundations of Java Development
BCA III Semester (Session 2026-2027)

Q1. Student Encapsulation [Easy] Create a class Student with private variables name, rollNo, and marks. • Create getter and setter methods for all variables. • Create displayDetails(). • Create one Student object. • Set values through setters and display values through getters. Concepts: private, getter, setter, data hiding.

```
class Student {
    private String name;
    private int rollNo;
    private double marks;

    public void setName(String name) {
        this.name = name;
    }

    public void setRollNo(int rollNo) {
        this.rollNo = rollNo;
    }

    public void setMarks(double marks) {
        this.marks = marks;
    }

    public String getName() {
        return name;
    }

    public int getRollNo() {
        return rollNo;
    }

    public double getMarks() {
        return marks;
    }

    void displayDetails() {
        System.out.println("Name: " + getName());
        System.out.println("Roll No: " + getRollNo());
```

```
        System.out.println("Marks: " + getMarks());
    }

    public static void main(String[] args) {
        Student s = new Student();

        s.setName("Sumit");
        s.setRollNo(101);
        s.setMarks(85.5);

        s.displayDetails();
    }
}
```

OUTPUT:
Name: Sumit
Roll No: 101
Marks: 85.5

Q2. Bank Account with Validation [Easy → Medium] Create a BankAccount class with private accountNumber, accountHolder, and balance. • Create getters and setters. • Create deposit(double amount). • Create withdraw(double amount). • Deposit must be greater than 0. • Withdrawal must not exceed the current balance. • Display appropriate messages for invalid transactions. Concepts: encapsulation, controlled access, validation.

```
class BankAccount {
    private int accountNumber;
    private String accountHolder;
    private double balance;

    void setAccountNumber(int n) { accountNumber = n; }
    void setAccountHolder(String n) { accountHolder = n; }
    void setBalance(double b) { balance = b; }

    int getAccountNumber() { return accountNumber; }
    String getAccountHolder() { return accountHolder; }
    double getBalance() { return balance; }

    void deposit(double amount) {
        if (amount > 0) balance += amount;
        else System.out.println("Invalid Deposit");
    }
```

```
    void withdraw(double amount) {
        if (amount > 0 && amount <= balance) balance -= amount;
        else System.out.println("Invalid Withdrawal");
    }

    public static void main(String[] args) {
        BankAccount b = new BankAccount();
        b.setAccountNumber(101);
        b.setAccountHolder("Sumit");
        b.setBalance(5000);

        b.deposit(1000);
        b.withdraw(2000);

        System.out.println("Holder: " + b.getAccountHolder());
        System.out.println("Balance: " + b.getBalance());
    }
}
```

OUTPUT:
Holder: Sumit
Balance: 4000.0

Q3. Employee Salary Encapsulation [Medium] Create an Employee class with private employeeId, employeeName, and salary. • Create getters and setters. • Salary cannot be negative. • Salary greater than 1,000,000 must also be rejected. • Display employee details. • Test at least one valid and one invalid salary. Concept: validation inside a setter

```
class Employee {
    private int employeeId;
    private String employeeName;
    private double salary;

    void setEmployeeId(int id) { employeeId = id; }
    void setEmployeeName(String name) { employeeName = name; }

    void setSalary(double s) {
        if (s >= 0 && s <= 1000000) salary = s;
        else System.out.println("Invalid Salary");
    }

    int getEmployeeId() { return employeeId; }
```

```
    String getEmployeeName() { return employeeName; }
    double getSalary() { return salary; }

    void display() {
        System.out.println(employeeId + " " + employeeName + " " + salary);
    }

    public static void main(String[] args) {
        Employee e = new Employee();

        e.setEmployeeId(101);
        e.setEmployeeName("Sumit");
        e.setSalary(50000);       // Valid
        e.setSalary(1200000);     // Invalid

        e.display();
    }
}
```

OUTPUT:
Invalid Salary
101 Sumit 50000.0

Q4. Animal → Dog [Easy] Create a parent class Animal containing eat() and sleep(). Create a child class Dog containing bark(). • Use extends. • Create a Dog object. • Call all three methods. Draw: Animal ↓ Dog Concept: Single inheritance.

```
class Animal {
    void eat() {
        System.out.println("Eating");
    }

    void sleep() {
        System.out.println("Sleeping");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Barking");
    }
```

```
}

public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.eat();
        d.sleep();
        d.bark();
    }
}
```

OUTPUT:
Eating
Sleeping
Barking

Q5. Person → Student [Easy → Medium] Create Person with name, age, and displayPerson(). Create Student extends Person and add rollNo, course, and displayStudent(). Create a Student object and display all information. Concepts: single inheritance and code reuse.

```
class Person {
    String name = "Sumit";
    int age = 19;

    void displayPerson() {
        System.out.println(name + " " + age);
    }
}

class Student extends Person {
    int rollNo = 101;
    String course = "BCA";

    void displayStudent() {
        displayPerson();
        System.out.println(rollNo + " " + course);
    }
}

public class Main {
    public static void main(String[] args) {
        Student s = new Student();
```

```
        s.displayStudent();
    }
}
```

OUTPUT:
Sumit 19
101 BCA

Q6. Employee → Manager with Encapsulation [Medium] Create an encapsulated Employee class with private name and salary and their getters/setters. Create Manager extends Employee and add department and displayManager(). Use inherited getter/setter methods to access Employee information. Do not make Employee fields public. Concepts: encapsulation + inheritance.

```
class Employee {
    private String name;
    private double salary;

    void setName(String n) { name = n; }
    void setSalary(double s) { salary = s; }
    String getName() { return name; }
    double getSalary() { return salary; }
}

class Manager extends Employee {
    String department = "IT";

    void displayManager() {
        System.out.println(getName() + " " + getSalary() + " " + department);
    }
}

public class Main {
    public static void main(String[] args) {
        Manager m = new Manager();
        m.setName("Sumit");
        m.setSalary(50000);
        m.displayManager();
    }
}
```

OUTPUT:

Sumit 50000.0 IT

Q7. Animal → Dog → Puppy [Easy] Create three classes: Animal → eat() Dog extends Animal → bark() Puppy extends Dog → play() Create a Puppy object and call all three methods. Draw the three-level hierarchy. Concept: multilevel inheritance.

```
class Animal {
    void eat() { System.out.println("Eating"); }
}

class Dog extends Animal {
    void bark() { System.out.println("Barking"); }
}

class Puppy extends Dog {
    void play() { System.out.println("Playing"); }
}

public class Main {
    public static void main(String[] args) {
        Puppy p = new Puppy();
        p.eat();
        p.bark();
        p.play();
    }
}
```

OUTPUT:
Eating
Barking
Playing

Q8. Person → Employee → Manager [Medium] Create: Person → name, displayName() Employee extends Person → employeeId, displayEmployee() Manager extends Employee → department, displayManager() Create a Manager object and display all information. Task: identify which member belongs to each class. Concept: multilevel inheritance.

```
class Person {
    String name = "Sumit";
    void displayName() { System.out.println(name); }
```

```
}

class Employee extends Person {
    int employeeId = 101;
    void displayEmployee() { System.out.println(employeeId); }
}

class Manager extends Employee {
    String department = "IT";
    void displayManager() {
        displayName();
        displayEmployee();
        System.out.println(department);
    }
}

public class Main {
    public static void main(String[] args) {
        Manager m = new Manager();
        m.displayManager();
    }
}
```

OUTPUT:
Sumit
101
IT

Q9. Vehicle → Car → ElectricCar [Medium] Create: Vehicle → start(), stop() Car extends Vehicle → drive() ElectricCar extends Car → chargeBattery() Create an ElectricCar object and demonstrate access to methods from all three levels. Concepts: multilevel inheritance and inherited behavior.

```
class Vehicle {
    void start() { System.out.println("Start"); }
    void stop() { System.out.println("Stop"); }
}

class Car extends Vehicle {
    void drive() { System.out.println("Drive"); }
}
```

```
class ElectricCar extends Car {
    void chargeBattery() { System.out.println("Charging"); }
}

public class Main {
    public static void main(String[] args) {
        ElectricCar e = new ElectricCar();
        e.start();
        e.drive();
        e.chargeBattery();
        e.stop();
    }
}
```

OUTPUT:
Start
Drive
Charging
Stop

Q10. Animal → Dog and Cat [Easy] Create: Animal → eat() Dog extends Animal → bark() Cat extends Animal → meow() Create objects of Dog and Cat. Demonstrate that both can use eat(). Draw the hierarchy. Concept: hierarchical inheritance.

```
class Animal {
    void eat() { System.out.println("Eating"); }
}

class Dog extends Animal {
    void bark() { System.out.println("Barking"); }
}

class Cat extends Animal {
    void meow() { System.out.println("Meowing"); }
}

public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        Cat c = new Cat();
```

```
        d.eat(); d.bark();
        c.eat(); c.meow();
    }
}
```

OUTPUT:
Eating
Barking
Eating
Meowing

Q11. Person → Student and Teacher [Medium] Create Person with name and displayName(). Create Student extends Person with course and study(). Create Teacher extends Person with subject and teach(). Create one Student and one Teacher object and display their details. Concept: hierarchical inheritance.

```
class Person {
    String name;
    void displayName() { System.out.println(name); }
}

class Student extends Person {
    String course = "BCA";
    void study() { System.out.println(course); }
}

class Teacher extends Person {
    String subject = "Java";
    void teach() { System.out.println(subject); }
}

public class Main {
    public static void main(String[] args) {
        Student s = new Student();
        s.name = "Sumit";
        s.displayName(); s.study();

        Teacher t = new Teacher();
        t.name = "Rahul";
        t.displayName(); t.teach();
    }
```

```
}
```

OUTPUT:
Sumit
BCA
Rahul
Java

Q12. Employee → Developer and Manager [Medium] Create Employee with employee name, employee ID, and displayEmployee(). Create Developer extends Employee with programmingLanguage and writeCode(). Create Manager extends Employee with department and conductMeeting(). Create both child objects. Identify inherited and child-specific members. Concept: hierarchical inheritance.

```
class Employee {
    String name = "Sumit";
    int id = 101;

    void displayEmployee() {
        System.out.println(name + " " + id);
    }
}

class Developer extends Employee {
    String language = "Java";
    void writeCode() { System.out.println(language); }
}

class Manager extends Employee {
    String department = "IT";
    void conductMeeting() { System.out.println(department); }
}

public class Main {
    public static void main(String[] args) {
        Developer d = new Developer();
        d.displayEmployee();
        d.writeCode();

        Manager m = new Manager();
        m.displayEmployee();
        m.conductMeeting();
    }
```

```
}
```

OUTPUT:
Sumit 101
Java
Sumit 101
IT

Q13. Animal Sound Overriding [Easy → Medium] Create Animal with makeSound(). Create Dog extends Animal and Cat extends Animal. Override makeSound() in both child classes. Dog should produce a barking message and Cat should produce a meowing message. Use @Override. Concept: method overriding.

```
class Animal {
    void makeSound() { System.out.println("Animal sound"); }
}

class Dog extends Animal {
    @Override
    void makeSound() { System.out.println("Barking"); }
}

class Cat extends Animal {
    @Override
    void makeSound() { System.out.println("Meowing"); }
}

public class Main {
    public static void main(String[] args) {
        new Dog().makeSound();
        new Cat().makeSound();
    }
}
```

OUTPUT:
Barking
Meowing

Q14. Employee Salary Calculation [Medium] Create Employee with calculateSalary(). Create Manager extends Employee. Override calculateSalary() in Manager using a different salary calculation. Create suitable objects and demonstrate which implementation executes. Concepts:

inheritance + method overriding.

```
class Employee {
    void calculateSalary() {
        System.out.println("Employee Salary: 30000");
    }
}

class Manager extends Employee {
    @Override
    void calculateSalary() {
        System.out.println("Manager Salary: 50000");
    }
}

public class Main {
    public static void main(String[] args) {
        Employee e = new Employee();
        Manager m = new Manager();

        e.calculateSalary();
        m.calculateSalary();
    }
}
```

OUTPUT:
Employee Salary: 30000
Manager Salary: 50000

Q15. Shape Area Overriding [Medium] Create Shape with calculateArea(). Create Circle and Rectangle as child classes. Override calculateArea() in both. Use appropriate formulas and display the areas. Use @Override. Concept: same method name with different child implementations.

```
class Shape {
    void calculateArea() {}
}

class Circle extends Shape {
    @Override
    void calculateArea() {
        System.out.println("Circle: " + 3.14 * 5 * 5);
    }
```

```
}

class Rectangle extends Shape {
    @Override
    void calculateArea() {
        System.out.println("Rectangle: " + 10 * 5);
    }
}

public class Main {
    public static void main(String[] args) {
        new Circle().calculateArea();
        new Rectangle().calculateArea();
    }
}
```

OUTPUT:
Circle: 78.5
Rectangle: 50

Q16. Camera + MusicPlayer → Smartphone [Medium] Create interfaces Camera and MusicPlayer. Camera: takePhoto() MusicPlayer: playMusic() Create Smartphone implements Camera, MusicPlayer and implement both methods.

Draw: Camera MusicPlayer
\ \ / /
Smartphone

Concept: multiple inheritance using interfaces.

```
interface Camera {
    void takePhoto();
}

interface MusicPlayer {
    void playMusic();
}

class Smartphone implements Camera, MusicPlayer {
    public void takePhoto() { System.out.println("Photo"); }
    public void playMusic() { System.out.println("Music"); }
}
```

```
public class Main {
    public static void main(String[] args) {
        Smartphone s = new Smartphone();
        s.takePhoto();
        s.playMusic();
    }
}
```

OUTPUT:
Photo
Music

Q17. Printer + Scanner → MultiFunctionMachine [Medium] Create interfaces Printer and Scanner. Printer: print() Scanner: scan() Create MultiFunctionMachine implementing both. Also use encapsulation for private machineName and machineId with getters/setters. Demonstrate both interface methods and encapsulated data. Concepts: multiple interfaces + encapsulation.

```
interface Printer { void print(); }
interface Scanner { void scan(); }

class MultiFunctionMachine implements Printer, Scanner {
    private String machineName;
    private int machineId;

    void setData(String n, int id) {
        machineName = n; machineId = id;
    }

    String getName() { return machineName; }
    int getId() { return machineId; }

    public void print() { System.out.println("Printing"); }
    public void scan() { System.out.println("Scanning"); }
}

public class Main {
    public static void main(String[] args) {
        MultiFunctionMachine m = new MultiFunctionMachine();
        m.setData("HP", 101);
        System.out.println(m.getName() + " " + m.getId());
        m.print();
```

```
        m.scan();
    }
}
```

OUTPUT:
HP 101
Printing
Scanning

Q18. Vehicle → Car → ElectricCar + Electric [Medium → Challenging]
Create Vehicle with start(). Create Car extends Vehicle with drive(). Create Electric interface with chargeBattery(). Create ElectricCar extends Car implements Electric. Create an ElectricCar object and call start(), drive(), and chargeBattery(). Draw the structure. Concepts: multilevel inheritance + interface implementation + hybrid structure.

```
class Vehicle {
    void start() { System.out.println("Start"); }
}

class Car extends Vehicle {
    void drive() { System.out.println("Drive"); }
}

interface Electric {
    void chargeBattery();
}

class ElectricCar extends Car implements Electric {
    public void chargeBattery() { System.out.println("Charging"); }
}

public class Main {
    public static void main(String[] args) {
        ElectricCar e = new ElectricCar();
        e.start();
        e.drive();
        e.chargeBattery();
    }
}
```

OUTPUT:
Start

Drive
Charging

Vehicle
  ↓
 Car ← Electric
  ↓
ElectricCar

Q19. Employee → Developer + Programmer + Researcher [Challenging] Create Employee with encapsulated name and employeeId and displayEmployee(). Create Programmer interface with writeCode(). Create Researcher interface with conductResearch(). Create Developer extends Employee implements Programmer, Researcher. Implement all required methods. Draw the hierarchy. Concepts: encapsulation, inheritance, multiple interfaces, hybrid structure.

```
class Employee {
    private String name;
    private int id;

    void setData(String n, int i) {
        name = n; id = i;
    }

    void displayEmployee() {
        System.out.println(name + " " + id);
    }
}

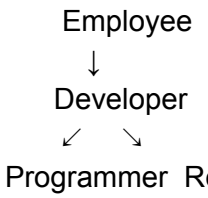
interface Programmer {
    void writeCode();
}

interface Researcher {
    void conductResearch();
}

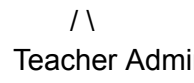
class Developer extends Employee implements Programmer, Researcher {
    public void writeCode() { System.out.println("Writing Code"); }
    public void conductResearch() { System.out.println("Researching"); }
}
```

```
public class Main {
    public static void main(String[] args) {
        Developer d = new Developer();
        d.setData("Sumit", 101);
        d.displayEmployee();
        d.writeCode();
        d.conductResearch();
    }
}
```

OUTPUT:
Sumit 101
Writing Code
Researching



Q20. University Employee System — Integrated OOP [Challenging] Design a university employee system. Hierarchy:



Also create Researcher interface. Employee: • private employeeId, employeeName, salary • getters and setters • displayDetails() • calculateSalary() Teacher: • subject • teach() • implements Researcher • override calculateSalary()

```
class Employee {
    private int id; private String name; private double salary;

    void setData(int i, String n, double s) { id=i; name=n; salary=s; }
    void displayDetails() { System.out.println(id+" "+name+" "+salary); }
    void calculateSalary() { System.out.println("Salary: "+salary); }
}

interface Researcher {
    void research();
```

```
}

class Teacher extends Employee implements Researcher {
    String subject="Java";
    void teach() { System.out.println("Teaching "+subject); }
    public void research() { System.out.println("Researching"); }
    @Override void calculateSalary() { System.out.println("Teacher Salary"); }
}

class Admin extends Employee {}

class VisitingTeacher extends Teacher {}

public class Main {
    public static void main(String[] args) {
        Teacher t=new Teacher();
        t.setData(101,"Sumit",50000);
        t.displayDetails();
        t.teach(); t.research(); t.calculateSalary();
    }
}
```

OUTPUT:

```
101 Sumit 50000.0
Teaching Java
Researching
Teacher Salary
```